

Python Tip: `*args` and `**kwargs`

Python makes it easy to create functions that can take any number of arguments. Use `*args` and `**kwargs` for flexible and reusable code.

Understanding * and **

Before diving into `*args` and `**kwargs`, let's understand the `*` and `**` operators.

The single asterisk (`*`) unpacks iterables like lists. The double asterisk (`**`) unpacks dictionaries.

```
# Single asterisk (*) unpacks an iterable
list1 = [1, 2, 3]
list2 = [4, 5, 6]

list3_with_unpacking = [*list1, *list2]
list3_without_unpacking = [list1, list2]

print(list3_with_unpacking)      # [1, 2, 3, 4, 5, 6]
print(list3_without_unpacking)
```

```
# Double asterisk (**) unpacks dictionaries
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}
```

```
dict3_with_unpacking = {**dict1, **dict2}
```

Using `*args` for Variable Arguments

When asterisks are used in function definitions, they pack arguments into tuples.

```
def average(*args): # args=(arg1, arg2, arg3, ..  
    return sum(args) / len(args)  
  
result = average(1, 2, 3, 4, 5)  
print(result) # 3.0
```

3.0

By packing arguments into a tuple, you can pass any number of arguments to the function.

Using `**kwargs` for Keyword Arguments

When double asterisks are used in function definitions, they pack keyword arguments into dictionaries.

```
def greet(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
greet(name="Alice", age=30)
```

```
name: Alice  
age: 30
```

`**kwargs` allows you to pass named arguments without explicitly defining each one.

Combining `*args` and `**kwargs`

You can use both `*args` and `**kwargs` in the same function.
Important: `*args` must appear before `**kwargs`.

```
def show_info(*args, **kwargs):  
    print("Positional arguments:", args)  
    print("Keyword arguments:", kwargs)  
  
show_info(1, 2, 3, name="Alice", job="Engineer")
```

```
Positional arguments: (1, 2, 3)  
Keyword arguments: {'name': 'Alice', 'job': 'Engi
```

Wrap-Up

Now you can create functions that handle any number of positional and keyword arguments, making your code more flexible and reusable.

Follow me for more tips.

Shep Bryan IV